Webshop alkalmazás készítése

Készítette: Gyáni Kevin Zsolt
Neptunkód: CBOYZF

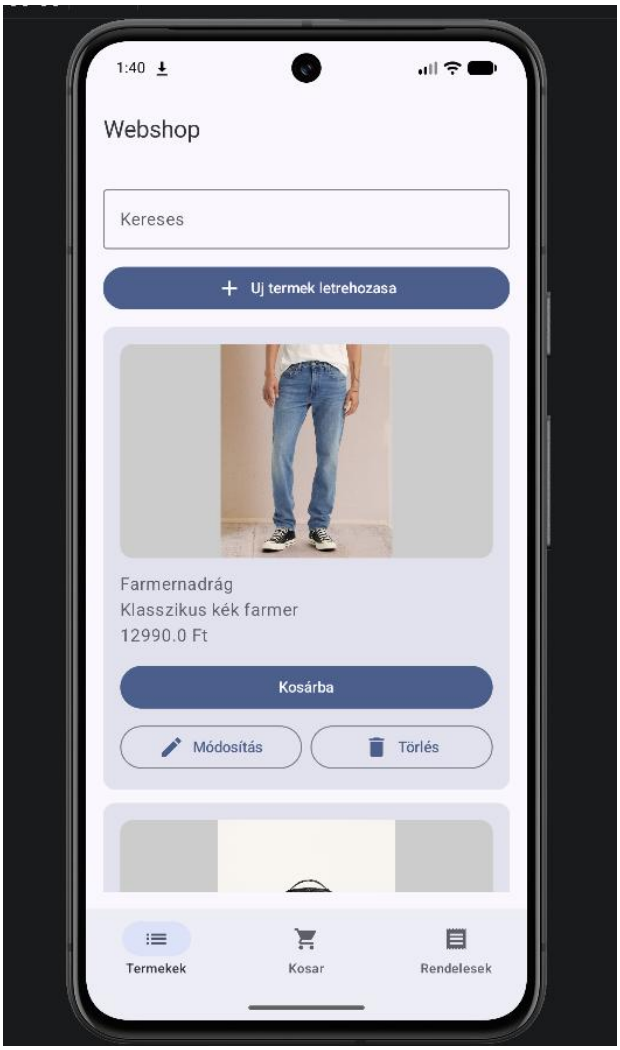
Tartalomjegyzék

A felület bemutatása: ... 3
Termékek: ... 3
Kosár: ... 5
Leadás, rendelések megtekintése ... 7
A kód bemutatása: ... 9
Adatstruktúra: ... 9
Az adatbázis: ... 10
A UI ... 11
Az alkalmazás (adminwebshopapp.kt) ... 11
ProductEditorDialog: létrehozás és szerkesztés: ... 12
Képkiválasztás működése: ... 13
ProductImagePreview: képmegjelenítés ... 13
AdminCartScreen: kosár kezelése ... 13
AdminOrdersScreen: rendelések listája ... 13

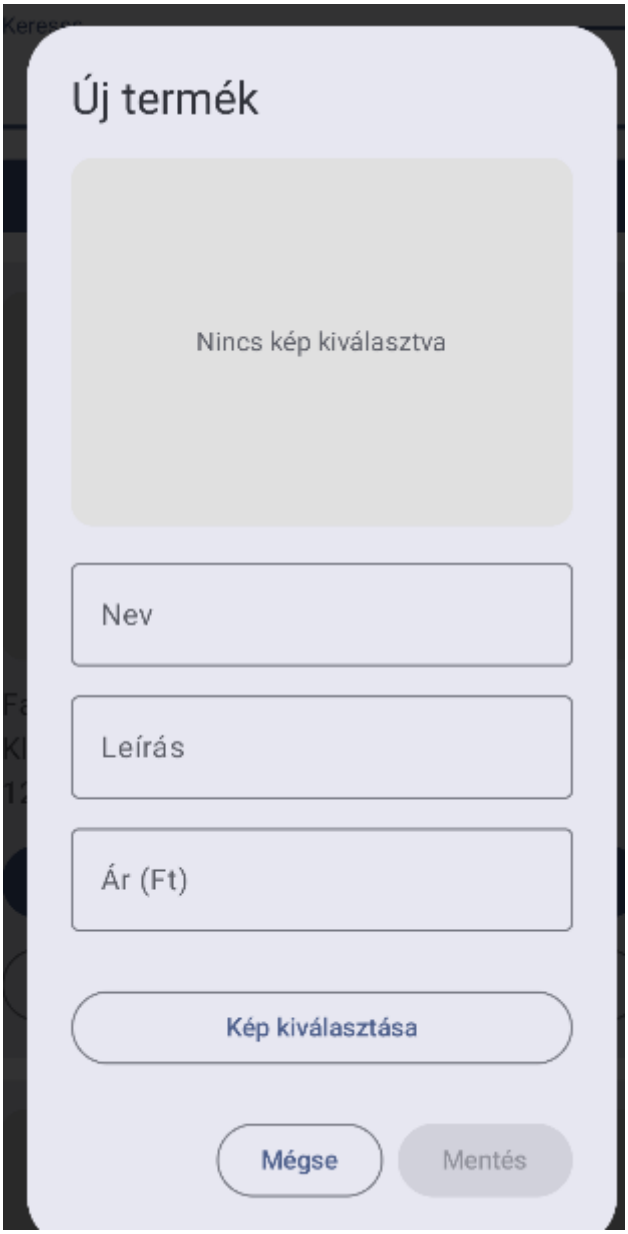
A felület bemutatása:

Termékek:

Az alkalmazás android studiobol történő indításakor az alábbi képernyő fogad:



A fenti ”Keresés” mező segítségével tudunk keresni a már feltöltött termékek között. A termékek egymás alatt egy legördülő listában jelennek meg. A kereső mező alatt elhelyezett ”Új termek létrehozása” gombra kattintva tudunk új terméket hozzáadni a webshop kínálatához.



A gombra kattintva a fenti dialógus nyílik, meg ahol megadhatjuk a feltölteni kívánt termék adatait. A termék képét a telefon tárhelyéről van lehetőségünk betallózni.

Ebből a képernyőből a Mégse gombra nyomva, vagy a dialógus mellé nyomva léphetünk, ki ha mégse kívánunk terméket hozzáadni.

A termékekkel 3 különböző műveletet van lehetőségünk elvégezni:
-Kosárhoz adás (erről majd később)
-Módosítás
-Törlés

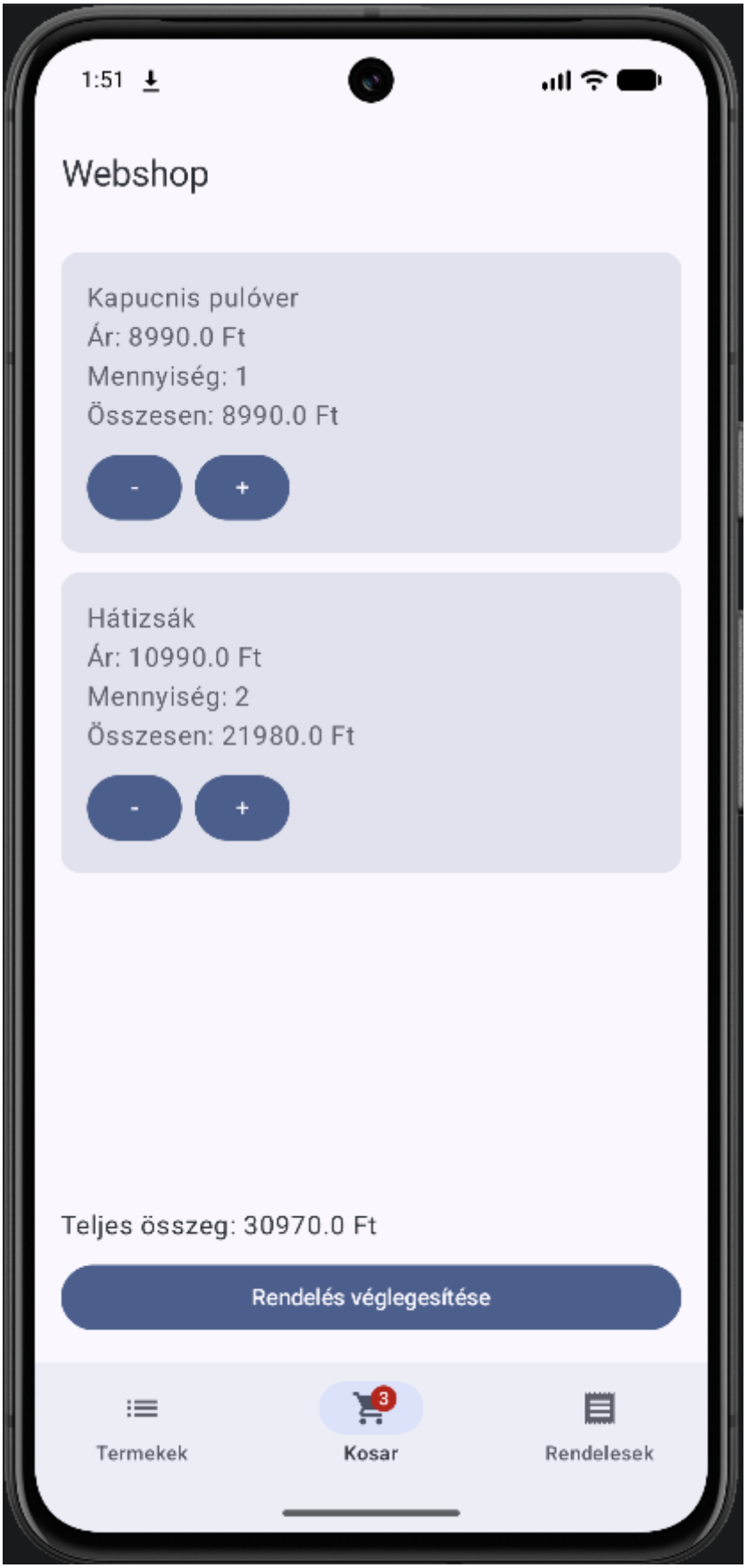
A módosítás gombra kattintva egy, az új termék létrehozásához ismerős ablak nyílik fel:

Itt lehetőségünk van a termék nevét, leírását, árát szerkeszteni, illetve a hozzá tartozó képet cserélni, valamint eltávolítani.

A törlés gomb törli az adott terméket a listából(kínálatból)

Kosár:

Amennyiben a Kosárba gombra nyomunk, a terméket a kosárhoz adhatjuk.

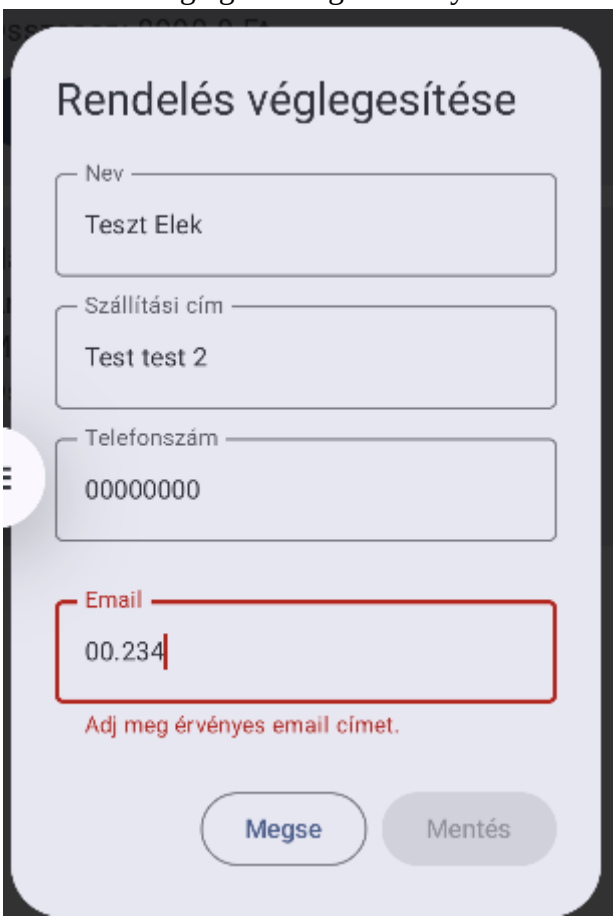


Miután minden kívánt terméket a kosárhoz adtunk a lenti „kosar” gombra/ikonra kattintva a

kosárhoz navigálhatjuk magunkat. Itt megjelen(nek) a kosárhoz adott termék(ek) illetve a mennyiségük, áruk , valamint össz áruk.

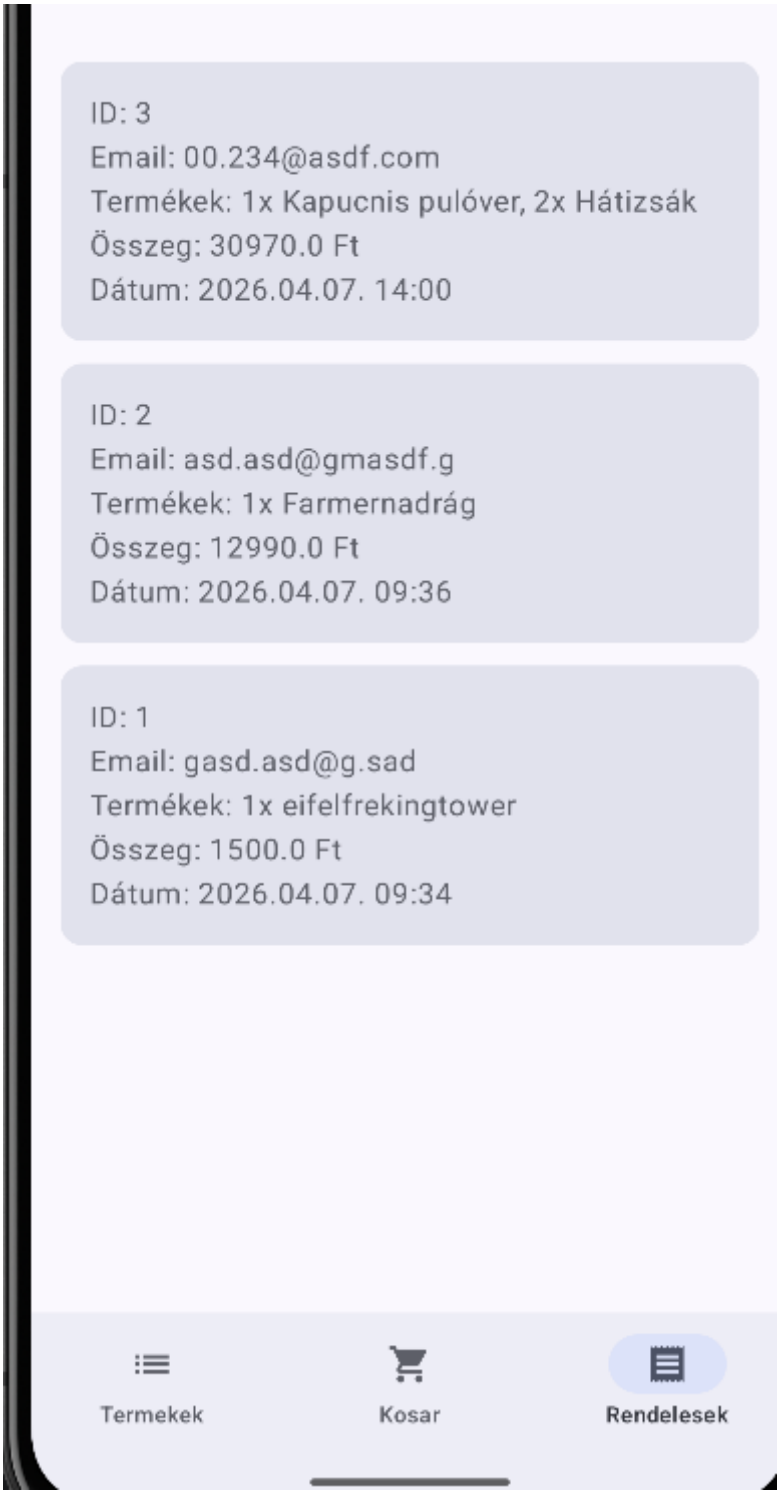
Leadás, rendelések megtekintése

A rendelés véglegesítése gombra nyomva a rendeléshez szükséges adatokat tölthetjük ki:



A telefonszám mezőbe kizárólag szám írható, illetve az email mező-be kizárólag érvényese emailcím formátum adható meg.

Érvényes emailcím formátum megadása után, a mentés gombra nyomva majd a Rendelések menüponthoz navigálva lehetőségünk van megtekinteni a már leadott rendelések részleteit.


ID: 3
Email: 00.234@asdf.com
Termékek: 1x Kapucnis pulóver, 2x Hátizsák
Összeg: 30970.0 Ft
Dátum: 2026.04.07. 14:00
ID: 2
Email: asd.asd@gmasdf.g
Termékek: 1x Farmernadrág
Összeg: 12990.0 Ft
Dátum: 2026.04.07. 09:36
ID: 1
Email: gasd.asd@g.sad
Termékek: 1x eifelfrekingtower
Összeg: 1500.0 Ft
Dátum: 2026.04.07. 09:34
Termekek
Kosar
Rendelesek

A kód bemutatása:

Adatstruktúra:

Az adatstruktúrának 2 fontos eleme van, az Entityk és a DAO-k, előbbik az adatbázis objektumokat reprezentálják, nagyon hasonló módon:

```
51 Usages
@Entity(tableName = "items")
data class ProductEntity(
    @PrimaryKey(autoGenerate = true) val id: Int = 0,
    val name: String,
    val description: String,
    val price: Double,
    val imageResName: String
)
```

A fentiekben egy Termék Entityje látható,

A @Entity annotáció jelenti, hogy ez egy Entity, a benne megadott tableName pedig az adatbázis tábla nevét adja, meg amire mutat. Ezt követően ugyan úgy definiáljuk, mint bármely más adatátvitelre szolgáló osztályt, azt leszámítva, hogy mivel adatbázis objektum így szükségünk van egy elsődleges kulcs megadására, amit a @PrimaryKey annotáció-val tehetünk meg, Ennek argumentuma az autoGenerate amit bekapcsolva megadhatjuk hogy termék létrehozásakor az adatbázis automatikusan generáljon egy ID-t (elsődleges kulcsot) az adott rekordnak. A többi entity megadás szinte pontosan ugyan így történik. (természetesen a tábla és változó nevek átírásával)

Az adatstruktúra részhez csoportosítanám még a DAO-t is (data access object).

A lentiekben a product-hoz megfeltetett DAO látható:

```
9 Usages  1 Implementation
@Dao
interface ProductDao {

    1 Usage  1 Implementation
    @Query( value = "SELECT * FROM items ORDER BY name ASC")
    fun getAllProducts(): Flow<List<ProductEntity>>

    1 Usage  1 Implementation
    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertAll(products: List<ProductEntity>)

    1 Usage  1 Implementation
    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insert(product: ProductEntity): Long

    1 Usage  1 Implementation
    @Update
    suspend fun update(product: ProductEntity)

    1 Usage  1 Implementation
    @Query( value = "SELECT COUNT(*) FROM items")
    suspend fun countProducts(): Int

    1 Implementation
    @Query( value = "DELETE FROM items")
    suspend fun deleteAllProducts()

    1 Usage  1 Implementation
    @Query( value = "DELETE FROM items WHERE id = :id")
    suspend fun deleteById(id: Int)
}
```

A Dao egy interface, azaz nem implementál metódusokat, az implementáció natív sql kódként az annotációkban történik. Kiemelendő ezek közül az OnConflictStrategy ami adatbázis konfliktus (pl már létező elsődleges kulcs,id, esetén felülírja az előzőt a jelen esetben)

Az adatbázis:

Az adatbázis implementálása a AppDatabase osztályban történik amely implementálja a RoomDatabase-t (ez is elegendő lenne az adatbázishoz, de ajánlott a dao-k használata) .

Itt a @Database annotáció adja, meg hogy az adatbázis mely entitykből épüljön fel.

```
15  @Database(
16      entities = [
17          ProductEntity::class,
18          CartItemEntity::class,
19          OrderEntity::class,
20          OrderItemEntity::class
21      ],
22      version = 4,
23      exportSchema = false
24  )
```

A verziót adott alkalmazáson belül kell emelni adatbázis változás esetében, hogy a program mindig a legfrissebb adatbázist használja. Az osztályban megtalálható még a getDataBase functon amely a databasebuilder segítségével létrehozza indításkor az adatbázist.

Ezt követően a WebshopRepository-abn történik az adatbázis seedelése, az a művelet, amikor egy üres adatbázist kezdeti adatokkal töltünk fel. Ezt én konstans adatokkal oldottam meg mivel a program funkcionalitásából adódóan folyamatosan változni fognak a használat közben. A termékek létrehozásához/eltovolításához a dao-ban megadott metódusokat használom.

A UI

A ui mappában 3 file található:

Color, ebben defeniálom a később használt színeket

Theme, a file elején definiálom a 2 témát (light,dark) a color-ban megadott színek felhasználásával.

Ezt követően egy funkcióban ellenőrzöm, hogy be van-e kapcsolva a dynamic color, ha igen akkor az ennek megfelelő színtár lesz használva, ellenkező esetben a saját előre téma kerül felhasználásra. Az ellenőrzést követően a MaterialTheme segítségével ráhúzom a témát a felhasználói elemekre (coloscheme,typhograph,content). A funkciót ellátam a @Composable annotációval is amely a Ui design elemek kirajzolásáért felelős.

Az alkalmazás (adminwebshopapp.kt)

Ebben a fileban történik a felhasználói felület összefoglalása a viewmodell alapján (erről később).

Az aktuálisan megnyitott képernyőt egy állapotváltozóban tároljuk, majd beolvassuk a viewmodellből az adatokat. A collectasstate miatt, ha valami változik a composable újra rajzolja az érintett részt. A Scaffold adja a képernyő keretét,
topbar: felső címsor
snack: rövid üzenetek küldés a felhasználónak az adott művelet sikerességéről a képernyő alján.
bottombar: alsó navbar

A NavigationBarItem-ek állítják át a currentScreen értékét, emiatt a 165-ös sorban található when blokk mindig a megfelelő képernyőt jeleníti meg. itt történik az összekötés is a ViewModel és az egyes UI-képernyők között:
onAddToCart = viewModel::addToCart

onCreateProduct = ...

Mivel a kisebb composek nem tudják, hogyan működik a db, csak callbacket hívnak.

Az AdminProductScreen-en több lokális UI-állapot van:

searchText: keresőmező tartalma

showEditor: nyitva van-e a létrehozó/szerkesztő dialógus

productToEdit: melyik terméket szerkesztjük

productToDelete: melyik terméket akarjuk törölni

A keresés logikája :
ha üres a keresőszöveg, minden termék látszik;

különben szűr név és leírás alapján;

remember(products, searchText) miatt csak akkor számolja újra, ha a terméklista vagy a keresőszöveg változik.

A LazyColumn a AdminWebshopApp.kt#L264 résznél jeleníti meg a szűrt termékeket, egyenként AdminProductCard formában.

A dialógus megnyitása/zárása itt történik:

ha showEditor == true, megjelenik a ProductEditorDialog;

ha szerkesztés történik, a productToEdit tartalmazza az adott terméket;

mentés vagy mégse után a dialógus bezáródik és az állapot nullázódik.

A törlés megerősítő ablak a AdminWebshopApp.kt#L304 körül van:

ha productToDelete != null, megjelenik az AlertDialog;

Torles esetén meghívja a törlő callbacket;

Megse esetén csak bezárja az ablakot.

Az AdminProductCard szerepe pusztán megjelenítés, ProductImagePreiew segítségével.

ProductEditorDialog: létrehozás és szerkesztés:

A ProductEditorDialog ugyanazt a párbeszédablakot használja új termék létrehozására és meglévő te rmék szerkesztésére is. A működését az initialProduct paraméter határozza meg: ha ez null, akkor új t ermék felviteléről van szó, ha pedig van benne egy termék, akkor
szerkesztési módban nyitjuk meg az ablakot.

A mezők értékeit a remember(initialProduct) segítségével tárolja, így szerkesztéskor a kiválasztott te rmék adatai automatikusan megjelennek az űrlapban. A mentés előtt alapvető ellenőrzés is történik: a program megvizsgálja, hogy a név és a leírás ki van-
e töltve, valamint hogy az ár mező valóban számmá alakítható-
e. Emiatt a Mentés gomb csak akkor lesz aktív, ha minden szükséges adat helyes formában rendelkez ésre áll.

Képkiválasztás működése:

A termékhez tartozó kép kiválasztását a rememberLauncherForActivityResult és az OpenDocument() Androidos beépített dokumentum választó biztosítja. Ennek segítségével a felhasználó a készülékéről tud képet megnyitni, amit az alkalmazás egy Uri (uniformResourceIdentifier) formájában kap vissza. A kiválasztás után a program megpróbál tartós olvasási jogosultságot kérni a fájlhoz, hogy később is hozzáférjen a képhez, majd az URI szöveges formában elmentésre kerül az imageSource változóban

ProductImagePreview: képmegjelenítés

A ProductImagePreview composable feladata, hogy a termékhez tartozó képet a megfelelő módon jelenítse meg. A függvény először eldönti, hogy az imageSource egy helyben kiválasztott képre mutató content:// URI, vagy egy alkalmazásba beépített drawable erőforrás neve. Ha URI-ról van szó, akkor a rendszer megpróbálja azt képpé alakítani, újabb Android-verziókon ImageDecoder, régebbi rendszereken pedig MediaStore segítségével. Ha nem külső képről van szó, akkor a program a getIdentifier(...) metódussal megkeresi a megfelelő drawable erőforrást. A megjelenítés végül háromféleképpen történhet: sikeresen betöltött helyi kép jelenik meg, sikeresen megtalált beépített kép jelenik meg, vagy ha egyik sem áll rendelkezésre, egy egyszerű helyettesítő felület látható szöveges visszajelzéssel.

AdminCartScreen: kosár kezelése

Az AdminCartScreen a kosár teljes kezeléséért felelős felületi komponens. Itt jelennek meg a kiválasztott termékek, azok neve, ára, mennyisége és soronkénti összértéke. A felhasználó a + és - gombokkal módosíthatja a termékek darabszámát, miközben a teljes összeg automatikusan újraszámolódik. Ugyanebben a képernyőben található a rendelés véglegesítéséhez szükséges űrlap is, amely név, cím, telefonszám és e-mail megadását kéri. A kód ellenőrzi, hogy a telefonszám csak számjegyekből álljon, az e-mail cím formailag helyes legyen, és minden kötelező mező ki legyen töltve. Ha az adatok megfelelőek, a felhasználó leadhatja a rendelést, a rendszer pedig meghívja a hozzá tartozó callbacket, majd kiüríti az űrlap mezöket és bezárja a párbeszédablakot.

AdminOrdersScreen: rendelések listája

Az AdminOrdersScreen a már leadott rendelések megjelenítésére szolgál. Ha még nincs egyetlen rendelés sem, akkor a képernyő egy rövid tájékoztató szöveget jelenít meg. Amennyiben vannak mentett rendelések, azokat egy LazyColumn listázza egymás alatt kártyás elrendezésben. Minden rendelésnél látható az azonosító, a vásárló e-mail címe, a megrendelt termékek összefoglalása, a teljes végösszeg, valamint a rendelés dátuma. A dátum megjelenítése külön segédfüggvényen keresztül történik, amely az időbélyeget olvasható formátumra alakítja át, így a rendelések áttekinthetően és felhasználóbarát módon jelennek meg.